

# Payment Service – Unit Testing Report

Flat Rental System (LuxeFlats) • Verification, Edge-Case Hardening & Defect Rectification

|                                       |                           |                             |                                |
|---------------------------------------|---------------------------|-----------------------------|--------------------------------|
| MICROSERVICE                          | TARGET SERVICE            | TEST SUITE                  | TEST FRAMEWORK                 |
| payment-service (Port 8084)           | MockPaymentService.java   | MockPaymentServiceTest.java | JUnit 5 / Mockito / Assertions |
| TEAM MEMBERS                          | JAVA / SPRING             | EXECUTION TOOL              | VERIFIED STATUS                |
| Harsh Sahu, Kshitij Goel, Tanya Gupta | Java 21 / Spring Boot 3.x | Maven Surefire Plugin 3.2.5 | 13 / 13 Passed (100%)          |

## A Objective

The Payment Service is a core microservice in the LuxeFlats rental platform. It manages the entire monetary transaction lifecycle for flat rentals: processing initial holding token deposits, monthly rental installments, enforcing idempotent payment rules, tracking transaction identifiers, and orchestrating tenant refunds.

The primary objectives of this unit-testing exercise were:

- Validate Core Payment Lifecycle:** Ensure valid token and rent payments succeed, generate a standardized reference (LF-TXN-...), transition to COMPLETED, and persist into the database.
- Enforce Payment Idempotency:** Confirm that the service prevents duplicate token payments on the same booking while correctly allowing subsequent distinct payment types such as RENT.
- Verify Refund Orchestration:** Ensure that refunds are only initiated for completed transactions and delegate execution cleanly to the underlying RefundService.
- Uncover Missing Validation & Authorization:** Actively probe for edge-case vulnerabilities—specifically zero and negative amounts, as well as cross-user refund attempts—that were absent from original code.

### Core Testing Philosophy

The purpose of unit testing in this project was not merely to achieve a green test counter or rubber-stamp code, but to actively probe the implementation for missing boundary validations and security gaps before deployment.

## B Existing Test Coverage (10 Baseline Tests)

Prior to our edge-case review, the Payment Service test suite contained **10 unit test cases** in MockPaymentServiceTest.java. These baseline tests verified standard service methods using Mockito mocks for PaymentRepository and RefundService:

| #  | Test Case Method                                  | Category      | Verification Purpose                                                                                                  | Result |
|----|---------------------------------------------------|---------------|-----------------------------------------------------------------------------------------------------------------------|--------|
| 1  | testCreatePayment_TokenSuccess                    | Token Payment | Creates a ₹5,000 token payment; asserts status COMPLETED, LF-TXN- prefix, and repository save() invocation.           | PASSED |
| 2  | testCreatePayment_DuplicateToken_ThrowsException  | Idempotency   | Prevents duplicate token payment on identical booking; throws PaymentProcessingException and never saves to database. | PASSED |
| 3  | testCreatePayment_RentPaymentAfterToken_Success   | Rent Payment  | Allows ₹25,000 RENT payment even if a TOKEN payment already exists for that booking (differentiates payment types).   | PASSED |
| 4  | testGetPaymentById_Success                        | Query         | Finds existing payment by primary key ID (1L); validates returned DTO fields and amount.                              | PASSED |
| 5  | testGetPaymentById_NotFound                       | Exception     | Queries non-existent payment ID (999L); confirms ResourceNotFoundException is thrown.                                 | PASSED |
| 6  | testGetPaymentsByBooking                          | Query         | Retrieves payment records filtered by bookingId (100L); validates collection size and booking mapping.                | PASSED |
| 7  | testGetPaymentsByTenant                           | Query         | Retrieves payment history for a tenant (200L); confirms proper tenant segregation.                                    | PASSED |
| 8  | testUpdateStatus_Success                          | Lifecycle     | Updates payment state to REFUNDED; verifies status change and persistence.                                            | PASSED |
| 9  | testInitiateRefund_Success                        | Refund Flow   | Initiates refund for completed payment; confirms status changes to REFUNDED and calls refundService.processRefund().  | PASSED |
| 10 | testInitiateRefund_PendingPayment_ThrowsException | Refund Guard  | Rejects refund when payment is still PENDING; throws PaymentProcessingException and prevents refund processing.       | PASSED |

## C Additional Test Cases Added (3 Edge-Case Tests)

During code inspection and testing, we recognized that while the baseline suite covered typical happy-path executions, it lacked tests for monetary boundary values and authorization boundaries. In financial microservices, accepting zero or negative numbers or failing to verify caller identity creates severe accounting and security risks.

To guarantee system robustness, **3 additional unit tests** were added to `MockPaymentServiceTest.java`:

1. **Zero Payment Amount (`testCreatePayment_ZeroAmount_ThrowsException`):**

*Why added:* In real rental workflows, a user submitting a payment request with `BigDecimal.ZERO` must be rejected immediately. Creating zero-value transaction records clutters ledgers and allows bookings to mark deposits as paid without actual funds.

2. **Negative Payment Amount (`testCreatePayment_NegativeAmount_ThrowsException`):**

*Why added:* Negative payment amounts (e.g. `-5000.00`) are classic input manipulation attacks. If unprocessed, they can reverse balance calculations, bypass invoice totals, or corrupt downstream accounting.

3. **Cross-User Refund Authorization (`testInitiateRefund_WrongUser_ThrowsException`):**

*Why added:* The refund method receives a `currentUserId`. If this parameter is not checked against the payment's associated `tenantId`, any tenant or malicious user knowing a payment ID could trigger a refund on another user's behalf.

## D Issues Found During Testing & Analysis

Executing the newly added test cases exposed two critical business logic defects in the original implementation of `MockPaymentService.java`:

### Defect 1: Missing Amount Boundary Validation in `createPayment()`

The initial implementation of `createPayment()` did not check `request.getAmount()`. Payments with zero (₹0.00) or negative (₹-5,000.00) amounts were accepted, assigned transaction reference numbers, marked as `COMPLETED`, and saved to the database.

### Defect 2: Missing Authorization Ownership Check in `initiateRefund()`

The `initiateRefund(Long paymentId, Long currentUserId)` method accepted `currentUserId` but never compared it against `payment.getTenantId()`. Any user could initiate a refund for another tenant's payment as long as the payment ID was valid.

### Test Construction Detail: Mockito Unnecessary Stubbing (Strict Mode)

During initial test drafting for zero and negative amounts, the test method copied an unused stub:

```
when(paymentRepository.findById(100L)).thenReturn(List.of());
```

Because our newly implemented validation logic correctly rejects zero/negative amounts at the very entrance of `createPayment()`, the execution never reached `findById()`. Mockito's strict runner (`@ExtendWith(MockitoExtension.class)`) flagged this unused stub as an `UnnecessaryStubbingException`.

*Clarification:* This was an internal test-suite stubbing refinement rather than an application bug. Removing the unnecessary repository stub from the test correctly asserted that boundary validation short-circuits execution before any database lookup.

E Code Fixes & Exception Handling

To resolve the discovered issues, targeted checks were implemented in MockPaymentService.java using consistent domain exceptions:

1. Added Amount Boundary Validation in createPayment():

```
public PaymentResponse createPayment(PaymentRequest request, Long tenantId) { // Added validation: reject null, zero,
and negative amounts before DB lookup if (request.getAmount() == null ||
request.getAmount().compareTo(BigDecimal.ZERO) <= 0) { throw new PaymentProcessingException("Payment amount must be
greater than zero."); } ... }
```

2. Added Tenant Authorization Ownership Check in initiateRefund():

```
public RefundResponse initiateRefund(Long paymentId, Long currentUserId) { Payment payment =
paymentRepository.findById(paymentId) .orElseThrow(() -> new ResourceNotFoundException("Payment not found with id: " +
paymentId)); // Added security check: caller must match the payment's associated tenant if
(!payment.getTenantId().equals(currentUserId)) { throw new PaymentProcessingException("You are not authorized to
refund this payment."); } ... }
```

Evolution of the 3 Additional Test Cases:

| Test Case                                                                  | Initial Observation                                                         | Fix Implemented                                                                                                | Final Result                                |
|----------------------------------------------------------------------------|-----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| <b>Zero Amount</b><br>testCreatePayment_ZeroAmount_ThrowsException         | Zero amount accepted; created a completed ₹0 payment record.                | Added compareTo(BigDecimal.ZERO) <= 0 guard throwing PaymentProcessingException. Removed unused test stubbing. | <div>PASSED</div> <div>0 DB writes</div>    |
| <b>Negative Amount</b><br>testCreatePayment_NegativeAmount_ThrowsException | Negative amount (₹-5,000) accepted without error; persisted invalid record. | Validated before repository query; rejected with PaymentProcessingException. Cleaned test stubbing.            | <div>PASSED</div> <div>0 DB writes</div>    |
| <b>Unauthorized Refund</b><br>testInitiateRefund_WrongUser_ThrowsException | User 999L was permitted to initiate refund for tenant 200L's payment.       | Added tenant comparison: !payment.getTenantId().equals(currentUserId) throwing PaymentProcessingException.     | <div>PASSED</div> <div>Refund aborted</div> |

## F Final Test Execution & Verified Results

Following code corrections, the full suite was executed via the Maven command line to verify isolation, assertion consistency, and zero regressions:

```
$ mvn test -pl payment-service [INFO] Scanning for projects... [INFO] -----< com.flatrental:payment-service >----- [INFO] Building payment-service 1.0-SNAPSHOT [INFO] -----[jar]----- [INFO] Running com.flatrental.payment.service.MockPaymentServiceTest [INFO] Tests run: 13, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 3.309 s -- in com.flatrental.payment.service.MockPaymentServiceTest [INFO] [INFO] Results: [INFO] Tests run: 13, Failures: 0, Errors: 0, Skipped: 0 [INFO] [INFO] ----- [INFO] BUILD SUCCESS [INFO] Total time: 28.713 s [INFO] -----
```

Verified Execution Summary

- **Total Unit Tests in Payment Service:** 13 tests (10 baseline + 3 edge-case additions)
- **Passed:** 13 | **Failures:** 0 | **Errors:** 0 | **Skipped:** 0
- **Pass Rate:** 100% test success rate
- **Execution Time:** 3.309 seconds (MockPaymentServiceTest)

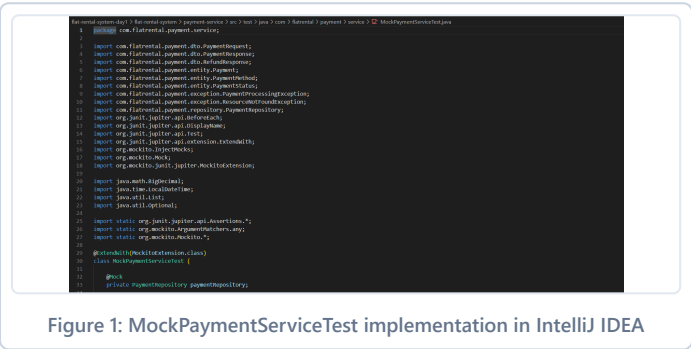


Figure 1: MockPaymentServiceTest implementation in IntelliJ IDEA



Figure 2: Payment Service unit test runner execution results (100% green)

## G Conclusion

The Payment Service unit-testing exercise succeeded in both validating baseline functionalities and uncovering critical real-world edge cases that were overlooked during initial feature coding.

Through systematic testing:

- The test suite grew from **10 to 13 robust unit tests**, achieving a verified **100% pass rate**.
- Input boundary vulnerabilities for zero and negative amounts were caught and resolved with early-exit validation, protecting financial consistency.
- A major cross-user refund authorization loophole was closed by enforcing caller-to-tenant verification.
- Tests were refined to adhere to strict Mockito best practices, eliminating brittle stubbing while accurately reflecting business execution flow.

The Payment Service is now significantly more resilient, secure, and ready for production deployment within the LuxeFlats platform.